CS275 Term Project Report

Athena Mo
University of California, Los Angeles
athenamo@g.ucla.edu

Victor Lee
University of California, Los Angeles
vlee9124@g.ucla.edu

Zhao Xu
University of California, Los Angeles
zhaohsu@g.ucla.edu

Ramnath Kumar
University of California, Los Angeles
ramnatthk@g.ucla.edu

Abstract

In this project, we explore multi-agent collaboration and using vision language models (VLM) as a communication layer. We present a multi-agent system and a VLM controlled agent built on top of the Unity ML-agents Pyramids reinforcement learning environment. We also explore the use of VLMs for direct locomotion and discuss its pitfalls and potential use as a planning layer. We also explore the use of curriculum learning as a training method pertaining to this sparse-reward environment.

ACM Reference Format:
Athena Mo, Victor Lee, Zhao Xu, and Ramnath Kumar. 2026. CS275 Term Project Report. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Multi-agent systems that must coordinate without a hand-specified protocol face a basic design question: what should agents tell each other, and in what representation? The standard answer in depp multi-agent reinforcement learning (MARL) is a learned communication channel - a real-valued vector exchange between policies which are optimized end-to-end alongside the task reward. This channel is expressive, but opaque: it carries no human-interpretable content, cannot be inspected mid-episode, and offers no obvious way to inject prior knowledge about the task. We ask whether a vision-language model (VLM), conditioned on the visual observations of multiple agents at once can serve as that channel instead - replacing learned vector with natural language scenes descriptions and sub-goals that are both interpretable, and easy to extend to new agents or new task structures.

We study this question on the Pyramids [4], a sparse-reward navigation task built in Unity ML-Agents in which an agent must find and press a hidden switch, wait for a gold pyramid to spawn, knock it over, and reach the gold brick beneath it. This four-step behavioral chain is rewarded only on completion (see Section 4.1). Pyramids is a natural testbed for this question: its long credit-assignment horizon makes it a demanding single-agent RL problem in its own right, and its behavioral chain decomposes naturally into roles that two agents could specialize in, with a VLM coordinating between them.

Our investigation proceeds along three threads that build towards a single system. Firstly (Section 2), we present our main contribution: rather than asking one VLM to perceive, plan and act, we split the Pyramids task across two role-specialized RL agents - one responsible for finding and pressing the switch, while the other for collecting the gold brick - trained jointly with Multi-Agent Posthumous Credit Assignment (MA-POCA [2]), and use a VLM as the communication layer between them. Because the VLM's context window holds the camera frames of both agents simultaneously, it can fuse cross-agent scene information. For instance, relaying that one of the agent is closer to the switch, and the other agent should just wait at an optimal cell with a long-horizon view is something neither agent's own egocentric view provides, without a hand-engineered messaging protocol. Secondly (Section 3), we ask a complementary question in isolation: can a VLM serve as a low-level action controller in its own right, replacing the learned policy entirely issuing discrete actions directly from egocentric camera frames? We find that it perceives the scene reliably, but cannot act on that perception at a useful rate - a 10-30 seconds decision latency against the task's real0time dynamics combined with stateless frame-by-frame querying makes it perceptually competent by mechanistically ill-suited to reactive control. Finally (Section 4), independent of the CLM, we ask whether curriculum learning can make single-agent training on this sparse-reward task more sample-efficient; a three-stage curriculum that exposes progressively more of the behavioral chain produces a markedly steeper learning curve than the baseline at matched compute.

Together, these threads motivate the design in Section 2: a VLM's strength in perception and high-level reasoning, and decomposing a sparse-reward task into specialized sub-problems is what makes it terrible in the first place. Our multi-agent system keeps low-level locomotion where it has always worked best - in a learned RL policy and elevates the VLM to the role its qualitative behavior actually supports a shared, language-mediated channel for coordinating two policies that would otherwise have no way to inform each other's decisions.

2 Multi-agent System

The original framework for Unity-ML Agents Pyramids is structured so that a single agent is tasked with three sequential sub-skills: locate and press a switch, navigate to the spawned gold pyramid, and reach the gold bricks of that pyramid. Instead, we recast this

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

framework to be a cooperative problem solved by two agents, each with independent and specialized roles.

The split of sub-skills between the two agents is designed so that neither agent can solve the task alone. To be specific, only one of the agents can flip the switch, and only the other can collect the gold. We train both agents jointly with Multi-Agent POsthumous Credit Assignment (MA-POCA) [3].

2.1 Task Decomposition

We instantiate a single multi-agent group containing two heterogeneous agents:

- *ButtonPresser* (π_B): tagged `buttonPresser`. Only this agent can flip the switch.
- *GoldCollector* (π_C): tagged `goldCollector`. Only this agent can end the episode by touching the gold brick.

We further enforced role lock at the level of Unity's collision callback rather than within the policy. The modified switch script returns early when the colliding object's tag is not `buttonPresser`, and only π_C's `OnCollisionEnter` reports a successful goal. Therefore, the two policies cannot collapse into a single dominant agent that solves the entire task, as the reward mechanism forces both agents to participate and collaborate.

Both agents inherit the four-action property of the baseline (rotate left/right, move forward/backward). Each agent independently observes the switch-state scalar, its own forward velocity, and a ray-cast field from three `RayPerceptionSensor3D` components. We extended the original sensor's detectable tag list to include the other agent's tag as well. This is so that each ray hit reports one extra channel that lets the agent localize its teammate when it is within line of sight. No privileged global observation (e.g., the other agent's absolute position) is provided.

2.2 Coordination via MA-POCA

We adopt the centralized training/decentralized-execution paradigm. During training, a single value function $V_{\text{team}}(s_B, s_C)$ takes the combined observations of both agents and is trained to predict the return of the shared team reward. At execution, each policy $\pi_B(a \mid s_B)$ and $\pi_C(a \mid s_C)$ receives only its own observations.

The shared team reward combines a small intermediate shaping term with the terminal goal:

$$r_t^{\text{group}} = \underbrace{0.5 \cdot \mathbb{1}[\text{switch flipped at } t]}_{\text{shaping}} + \underbrace{2.0 \cdot \mathbb{1}[\text{gold reached at } t]}_{\text{terminal}}, \tag{1}$$

where each agent additionally receives an individual time penalty $r_t^i = -1/T_{\max}$ as in the baseline. Since touching the gold bricks is the only signal that terminates an episode, the team is naturally incentivized to

3 Vision Language Model (VLM) Action Layer

In a new Pyramids environment, we replace the agent's trained PPO network controller with a Vision Language Model (VLM) and ask it to act as a zero-shot, training free control policy where at each decision step, the model receives the agent's egocentric camera frame and the task instruction via a natural language prompt, and returns the next discrete action directly. Similar to the RL agent, the VLM controller is given the same task and same four action discrete action set and is expected to solve it using only visual and guiding information baked into the prompt.

Figure 1: Diagram of VLM Agent Architecture

Our findings show that the VLM does perceive the scene, reason about it, and issues sensible commands, but overall is inefficient in being a low level action controller. The bottleneck is not due to competence, but rate due to the multi-billion parameter model's inference speed not closing the reactive control loop that locomotion requires, and the stateless frame by frame querying we adopt also discards temporal continuity a navigator needs to fully understand a scene (Sections 3.3-3.4).

3.1 VLM Agent Architecture

The VLM controller works as a three-stage pipeline that connects the running Unity environment to a locally hosted VLM from Ollama, and feeds the model's inference back into the bridge which parses the response as a discrete action (Figure 1).

Integration via heuristic channel. ML-Agents exposes a `Heuristic()` hook intended for scripted or human control. We route the VLM through it, replacing the learned policy rather than augmenting it, using the same 4 + 1 discrete actions – rotate left/right, move forward/backward, or no-op.

Perception. The agent's observation mechanism is a egocentric forward facing camera ($\approx$ 100° field of view) mounted at the agent. We use the Unity camera object which captures a 384x384px JPEG (quality 70) image (Figure 2) which is then encoded into a base64 string. Compared to the PPO agent which receives the 148-dimension ray-cast vector, the VLM agent only receives the image from which it must figure out the spatial structure of the map (the corridors, walls, switch, white pyramids, and gold pyramid).

Control Loop. Because inference is slow, the planner cannot keep up with the simulation. Instead, it runs asynchronously: a request is issued roughly every 2 seconds, and each returned action is cached and played. The simulator and VLM thus advance at very different rates, with the cache attempting to bridge the gap.

Local inference backend. The Unity controller does not directly communicate with the model; it POSTs the JSON request to a small HTTP bridge, which reformats it for the inference server and relays the structured reply. We run the open-source Qwen2.5-VL (7B) hosted using Ollama. The entire stack is local and offline requiring no external API calls so the latency figures we report reflect on-device performance on the Apple M2 MacBook Pro 16GB rather than network trips. The bridge also records request/response pairs, which we use for a live monitoring dashboard and a source of latency measurements in Section 3.3.

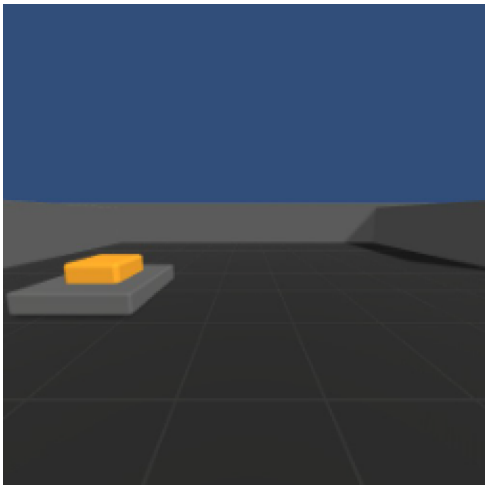

Figure 2: 384x384px view taken from the Agent's front facing camera.

3.2 Structured Actions and Prompting

The VLM's response is constrained to a single JSON object per frame whose fields separate what to do from why: a high-level `command` (`seek_switch`, `seek_goal`, `search`, or `avoid_obstacle`), a low-level `action` drawn from the agent's discrete set, a `confidence` score, a `command_ttl_seconds` lifetime, and two natural-language fields—a `scene_description` and a `reasoning` string. The prompt instructs the model to populate these in order, describing the visible corridors, walls, switch, white pyramids, and gold brick before committing to an action; this scene-first ordering acts as a lightweight chain-of-thought that grounds the chosen move in stated evidence. We pair this with a handful of appearance hints for the objects the model most often misses (e.g. the low, easily-overlooked switch pad and the white/gray pyramid stacks that blend into the walls). We decode at temperature = 0 with JSON-constrained output so that decisions are machine-parseable.

We require a per-frame action for our action controller model, but the model also volunteers a command that describes higher-level intent at the granularity of a sub-goal (e.g."seek the switch," "search the room") which persists naturally across many frames. In practice the command is the more stable and reliable half of the response, while the low-level action is where the model falters (Section 3.4).

3.3 Results

We evaluate the controller qualitatively, by observing agent behavior across episodes, and quantitatively along the axis that dominates a real-time controller: decision latency. Running Qwen2.5-VL (7B) locally under Ollama on an Apple M2 MacBook Pro 16GB, a single decision—camera capture, transfer, inference, and parsing—takes an observed 10–30 s end-to-end. The consequences for control compound. The resulting effective control rate—on the order of [0.03–0.1 Hz]—sits three to four orders of magnitude below Unity's default 50 Hz physics simulation, so the agent spends the overwhelming majority of every episode motionless, waiting on the model.

Qualitatively, perception is the strong half of the system: the model's `scene_description` reliably names corridors and walls and, when in view, the switch and gold brick. Acting on that understanding is where it breaks down. The agent frequently wedges itself against walls and corners, repeating the same action (e.g. turning left into a wall, or driving forward into one) across many decisions, and the 10–30 s latency turns every correction into overshoot: a decision computed on one frame is applied against a scene the agent has long since drifted past. It often misses the switch even with the pad plainly in view, naming it in the scene description yet failing to steer to it. These are the signatures of a controller whose feedback loop is far slower, and far less reliable, than the dynamics it must control.

3.4 Discussion of VLMs for Action Controllers

Two independent limitations, not one, make a frozen VLM unsuitable as a low-level controller. Latency dominates control. A 10–30 s decision against a 50 Hz simulation is a three-to-four order-of-magnitude rate mismatch; the cache and decoupled poll loop of Section 3.2 are not features but workarounds that keep the system running while leaving the agent frozen for all but a few percent of each episode. No amount of prompt engineering closes a gap this large; reactive locomotion needs reflexes on the order of milliseconds, and a multi-billion-parameter forward pass cannot supply them sufficiently. Grounding fails independently of speed. Even setting latency aside, the model that correctly names the switch in its `scene_description` still fails to steer toward it: it perceives what is there far more reliably than it can decide what to do about it, and that gap is exactly what the symbolic guardrail of Section 3.4 was written to patch. A controller that both acts at 0.05 Hz and cannot reliably convert perception into motor commands is the wrong tool for the job on two counts.

A third, quieter limitation compounds both: the controller is stateless. Each request is independent, so the agent has no memory with which to recover from the overshoot and oscillation that latency induces. One possible direction is to integrate memory and a virtual environment map

From the results, we observe the VLM's reliable competence is perceptual and high-level, while the action layer demands fast, stateful, well-grounded motor control.

4 Curriculum Learning for Sparse-Reward Exploration

Before converging on the multi-agent formulation that constitutes our main method, we explored *curriculum learning* as a way to make single-agent training on the Pyramids task substantially more sample-efficient. While this study is not part of our final system, it isolates a clean lesson—that decomposing a sparse-reward task into a sequence of progressively harder sub-tasks yields a markedly steeper learning curve—and it directly motivates the adaptive, VLM-driven sub-goal generation we discuss as future work (Section 4.6).

4.1 Problem Setting

Pyramids [4] is a sparse-reward navigation task in Unity ML-Agents. The agent observes a 148-dimensional vector of local ray-casts that detect the switch, stone pyramids, the gold brick, and walls, plus

Table 1: The three-stage curriculum. `task_difficulty`= 0 reproduces the baseline task exactly, making the comparison apples-to-apples.

Lesson	`task_difficulty`	What the agent sees
L0 – *GoldOnly*	2.0	Gold pyramid pre-spawned *and* switch pre-pressed; reduces to a pure navigate-to-goal task.
L1 – *ButtonNear*	1.0	Switch placed in the agent's own spawn cell; the agent must press it to spawn the pyramid, but need not search for it.
L2 – *Full*	0.0	Standard Pyramids task; switch randomly placed across the 3×3 grid. Identical to the baseline.

a scalar indicating the switch state, and it chooses among four discrete actions (rotate left/right, move forward/backward). The reward is

$$r_t = \underbrace{+2}_{\text{gold brick reached}} - \frac{1}{T_{\max}} \quad \text{per step,} \tag{2}$$

i.e. the agent receives a large terminal reward of +2 only after completing the full behavioral chain—*find and press the switch* → *a gold pyramid spawns* → *knock it over* → *reach the gold brick*—and a small per-step penalty otherwise. We train with PPO [8] augmented by an Intrinsic Curiosity Module (ICM) [6], the default exploration bonus shipped with ML-Agents.

Because no intermediate reward decomposes the task, an agent learning from scratch spends the large majority of early training timing out at $r \approx -1$ (Figure 3, orange). The terminal reward is so rarely encountered that policy improvement is slow and high-variance.

4.2 Curriculum Design

We introduce a single scalar environment parameter, `task_difficulty`, that selects one of three lessons, ordered from easiest to hardest (Table 1). Each lesson exposes a strictly larger portion of the behavioral chain in Equation (2), so that a sub-skill acquired in one lesson transfers as a useful prior to the next.

Automatic lesson advancement. We use ML-Agents' native curriculum API rather than a fixed schedule. The agent advances from lesson ℓ to $\ell + 1$ once the smoothed mean episode reward in the current lesson exceeds a threshold τ:

$$\tilde{r}^{(\ell)} \geq \tau = -0.5 \quad \text{over a minimum of 100 episodes,} \tag{3}$$

where $\tilde{r}^{(\ell)}$ is the exponentially smoothed mean reward. The threshold $\tau = -0.5$ is deliberately lenient: it fires as soon as the agent is *better than a guaranteed timeout*, rather than waiting for mastery, which keeps the agent moving through the curriculum while each sub-skill is still improving.

Fairness of comparison. All PPO and ICM hyperparameters are held identical between the baseline and the curriculum runs (Table 2); the two configurations differ *only* in the presence of the `environment_parameters` block. Since `task_difficulty`= 0 recovers the baseline task, any performance gap is attributable to the curriculum schedule alone.

4.3 Implementation

The curriculum required two small, self-contained code changes, both guarded by `task_difficulty` so that the default value 0 leaves baseline behavior unchanged. In `PyramidAgent.OnEpisodeBegin`, we read the parameter and branch on the lesson:

```
float taskDifficulty = Academy.Instance.EnvironmentParameters
    .GetWithDefault("task_difficulty", 0f);

// L1: spawn the switch in the agent's own cell; L0/L2: random
      across grid.
int switchSpawnIdx = (taskDifficulty > 0.5f && taskDifficulty < 1.5
      f)
    ? items[0] : items[1];
m_SwitchLogic.ResetSwitch(switchSpawnIdx, items[2]);
// ... stone pyramids created as in the baseline ...
if (taskDifficulty > 1.5f) { // L0: pre-spawn gold + press switch
    m_MyArea.CreatePyramid(1, items[2]);
    m_SwitchLogic.PressSwitch();
}
```

A new `PyramidSwitch.PressSwitch()` method marks the switch as on (state, tag, and material) *without* triggering the usual pyramid spawn, since in lesson L0 the gold pyramid is created externally. The curriculum schedule itself is declarative, expressed entirely in the training configuration:

```
environment_parameters:
  task_difficulty:
    curriculum:
      - name: Lesson0_GoldOnly
        completion_criteria: {measure: reward, threshold: -0.5,
                              min_lesson_length: 100, signal_smoothing:
                                    true}
        value: 2.0
      - name: Lesson1_ButtonNearAgent
        completion_criteria: {measure: reward, threshold: -0.5,
                              min_lesson_length: 100, signal_smoothing:
                                    true}
        value: 1.0
      - name: Lesson2_Full
        value: 0.0
```

4.4 Experimental Setup

We compare a baseline (no curriculum) against the three-stage curriculum, each trained for 1.5M environment steps with the hyperparameters in Table 2. Both runs use a single seed on an Apple M2 MacBook Pro (16 GB); each run takes roughly 75 minutes. A preliminary 500k-step sweep (discussed in Section 4.6) was used only to set the advancement threshold and lesson count and is not used for the headline comparison. We report the mean cumulative reward over a sliding window—the primary metric, fully reproducible from the saved scalar logs—and the episode *win rate* (fraction of episodes that terminate by reaching the gold brick, i.e. a positive task return) as a complementary, more interpretable measure.

4.5 Results

Figure 3 shows the training curves. The curriculum advances through its lessons early—L0→L1 at step 340k and L1→L2 at step 430k—leaving roughly 1.07M steps of full-task ($\ell = 2$) training, the same task the baseline trains on throughout. After entering the full task

Table 2: Shared PPO/ICM hyperparameters. Baseline and curriculum runs are identical except for the `environment_parameters` curriculum block.

Hyperparameter	Value
Trainer / algorithm	PPO
Batch size / buffer size	128 / 2048
Learning rate (schedule)	3×10^{-4} (linear)
β / ϵ / λ	0.01 / 0.2 / 0.95
Epochs per update	3
Hidden units / layers	512 / 2
Extrinsic γ / strength	0.99 / 1.0
Curiosity (ICM) strength	0.02
Time horizon	128
Max steps	1.5×10^6

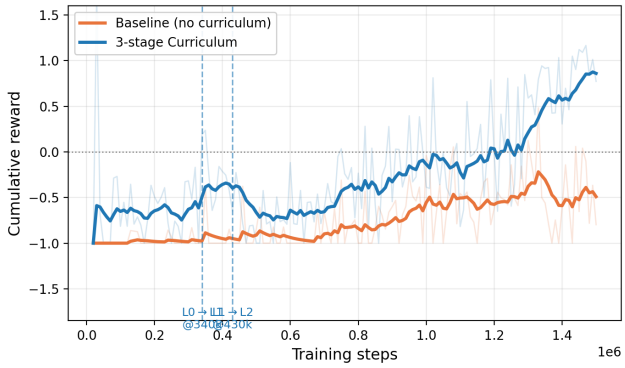

Figure 3: Cumulative reward over 1.5M training steps (faint: raw; bold: exponentially smoothed). The baseline (orange) plateaus near −0.5, whereas the curriculum (blue) shows two inflection points at the lesson transitions—L0→L1 at 340k and L1→L2 at 430k steps—and then climbs steadily into positive reward.

the curriculum climbs steadily, while the baseline stays flat near −0.5.

Table 3 quantifies the gap. Over the final 100k steps the curriculum reaches a mean reward of +0.86—it succeeds more often than it times out—versus −0.45 for the baseline. In terms of win rate this is a jump from 18.4% to 61.9%, a 3.4× relative improvement at matched compute. Crucially, the advantage *compounds* rather than shifting by a constant: the windowed mean reward (Figure 4) widens from a 0.36 gap at 400–500k to a 1.31 gap at 1.4–1.5M, confirming the curriculum agent is on a fundamentally steeper learning curve.

4.6 Discussion, Limitations, and Connection to the Main Method

Lesson design matters more than threshold tuning. Our 500k-step pilot found that a naive two-stage curriculum—jumping directly from "no switch needed" to "find a random switch"—gave almost no benefit (+1.4 percentage points). Inserting the intermediate L1 sub-skill (*press a nearby switch*) is what produced the large gain,

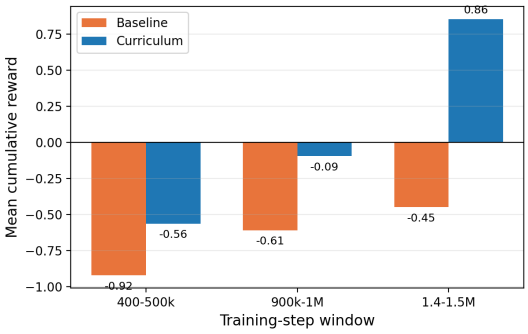

Figure 4: Mean cumulative reward at three step windows. The gap does not merely shift but *widens*; by the final window the curriculum agent has crossed into positive reward (net success) while the baseline remains negative (net timeout).

Table 3: Baseline vs. three-stage curriculum at 1.5M steps. Mean-reward figures are computed directly from the training scalar logs; win rate is the fraction of episodes reaching the gold brick in the final window.

Metric (final 100k steps)	Baseline	Curriculum
Mean cumulative reward	−0.45	**+0.86**
Win rate	18.4%	**61.9%**
Mean reward by step window		
400–500k	−0.92	−0.56
900k–1M	−0.61	−0.09
1.4–1.5M	−0.45	+0.86

consistent with the view that the value of a curriculum lies in how finely it decomposes the skill hierarchy [1, 5], not in the advancement threshold.

Limitations. The headline result uses a *single seed*; ML-Agents PPO has high seed variance (an identical configuration triggered L0→L1 anywhere between 95k and 340k steps across pilot runs), so a publication-grade claim would require ≥ 3 seeds with mean±std. The 1.5M-step budget is also only ∼15% of the canonical 10M-step Pyramids benchmark, so the result is best read as a strong advantage *at matched compute* rather than an absolute state-of-the-art number. Finally, the L1 design—placing the switch in the agent's spawn cell—reduces switch *distance* but not switch *detection* difficulty (six stone-pyramid distractors remain), and cleaner decompositions are possible.

Connection to the main method. This hand-designed curriculum is a static stand-in for what an adaptive cognition layer could provide automatically. Two directions follow. First, an *automatic* curriculum (e.g. ALP-GMM teacher algorithms [7]) could select the next lesson from the agent's measured learning progress, removing the manual lesson-design step. Second—and the reason this study lives in our report—a vision-language model acting as a cognition layer could propose adaptive sub-goals from the rendered scene, effectively generating a curriculum on the fly. The manual three-stage

schedule here is the simplest realization of that idea, and it bridges directly to the VLM-driven, multi-agent system that constitutes our main contribution.

5 Future Work

Acknowledgments

We thank the instructor of UCLA's CS275 Artificial Life Demetri Terzopoulos and the Unity ML Agent's contributors and maintainers for making this project possible.

References

[1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum Learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*. 41–48.

[2] Andrew Cohen, Ervin Teng, Vincent-Pierre Berges, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook, and Sujoy Ganguly. 2021. On the use and misuse of absorbing states in multi-agent reinforcement learning. *arXiv preprint arXiv:2111.05992* (2021).

[3] Andrew Cohen, Ervin Teng, Vincent-Pierre Berges, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook, and Sujoy Ganguly. 2022. On the Use and Misuse of Absorbing States in Multi-Agent Reinforcement Learning. In *AAAI Workshop on Reinforcement Learning in Games*.

[4] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. 2018. Unity: A General Platform for Intelligent Agents. *arXiv preprint arXiv:1809.02627* (2018).

[5] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. 2020. Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey. *Journal of Machine Learning Research* 21, 181 (2020), 1–50.

[6] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. 2017. Curiosity-Driven Exploration by Self-Supervised Prediction. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2778–2787.

[7] Rémy Portelas, Cédric Colas, Katja Hofmann, and Pierre-Yves Oudeyer. 2020. Teacher Algorithms for Curriculum Learning of Deep RL in Continuously Parameterized Environments. In *Conference on Robot Learning (CoRL)*. 835–853.

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).

A Online Resources

Unity ML Agents

We perform most of our work using the Unity ML Agents framework which integrates Unity virtual environments and training tools.

Unity ML Agents Documentation [Link]

The example we base the work on is Unity Ml Agents Pyramids explained here:

ML Agents Example Environment -Pyramids

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009